

SQLite Parser

소스 분석

토큰 · 문법 규칙 · 이름 해석 · 매개변수

Week 3-2 데이터베이스
1분반 9/18 · 2분반 9/17

03

SQLite
소스 분석

분석 환경과 실행

별도 Parser 패키지에서 실행합니다.

```
python3 setup.py  
./build/token_probe  
./build/student_lab data/week3.db parser ok
```

Python 3.10 이상과 C 컴파일러 cc가 필요합니다.

지원 환경: macOS, Linux, Windows WSL

분석할 소스 파일

파일	이번 시간의 관찰 지점
src/prepare.c	공개 prepare API에서 파서까지의 호출
src/tokenize.c	토큰 추출과 Parser에 전달하는 반복문
src/parse.y	Lemon 문법 규칙과 C 의미 동작
src/expr.c	매개변수의 번호 결정
src/vdbeapi.c	바인딩 값을 실제로 저장하는 위치

경로 기준: vendor/sqlite-src-3530400/

prepare에서 Parser까지

순서	함수	파일과 시작 행
1	sqlite3_prepare_v2	prepare.c:937
2	sqlite3LockAndPrepare	prepare.c:836
3	sqlite3Prepare	prepare.c:682
4	sqlite3RunParser	tokenize.c:600

sqlite3Prepare 안의 779행 또는 786행에서 sqlite3RunParser를 호출합니다.

컴파일과 실행의 구분

prepare 시점

토큰화, 문법 처리, 이름 해석과 실행 코드 준비를 수행합니다.

bind 시점

준비된 문장의 매개변수 저장소에 값을 넣습니다.

step 시점

준비된 문장을 실행하여 결과 행 또는 완료 상태를 얻습니다.

v2 문장은 스키마 변경 등의 조건에서 step 중 자동 재컴파일될 수 있습니다.

토큰라이저의 실제 출력

token_probe가 sqlite3GetToken의 결과를 출력합니다.

바이트 위치	길이	토큰 종류	문자열
0	6	TK_SELECT	SELECT
7	4	TK_ID	name
12	4	TK_FROM	FROM
36	2	TK_VARIABLE	?1
38	1	TK_SEMI	;

기본 SQL: SELECT name FROM student WHERE id = ?1; 공백 등 일부 행 생략

?1을 인식하는 코드

tokenize.c:504-508의 실제 코드입니다.

```
case CC_VARNUM: {
    *tokenType = TK_VARIABLE;
    for(i=1; sqlite3Isdigit(z[i]); i++){
        return i;
    }
}
```

물음표 뒤의 숫자를 읽고, 토큰 종류와 바이트 길이를 돌려줍니다.

여기서는 값 2001을 읽지 않습니다. SQL 안의 문자 ?1만 읽습니다.

토큰을 Parser에 전달하는 반복문

tokenize.c:646과 711-715를 연결해 읽습니다.

```
n = sqlite3GetToken((u8*)zSql, &tokenType);  
/* 길이, 공백, 오류 처리 분기는 생략 */  
pParse->sLastToken.z = zSql;  
pParse->sLastToken.n = (u32)n;  
sqlite3Parser(pEngine, tokenType, pParse->sLastToken);  
zSql += n;
```

원문 위치와 길이를 함께 넘기고, 다음 토큰으로 이동합니다.

발췌 사이에 공백 처리, 토큰 오류 검사, 중단 조건 등이 있습니다.

Lemon과 parse.y

문법의 원본

src/parse.y에 SQL 문법과 C 의미 동작을 정의합니다.

생성 도구

tool/lemon.c와 tool/lempar.c로 Parser 코드를 생성합니다.

읽을 대상

이번에는 생성된 거대한 코드보다 원본 문법 규칙을 읽습니다.

배포 amalgamation에는 해당 릴리스에서 생성한 Parser 코드가 이미 포함됩니다.

SELECT 문법 규칙

parse.y:651-655에서 규칙과 의미 동작을 구분합니다.

```
oneselect(A) ::= SELECT distinct(D) selcollist(W) from(X)
                where_opt(Y) groupby_opt(P) having_opt(Q)
                orderby_opt(Z) limit_opt(L). {
    A = sqlite3SelectNew(pParse,W,X,Y,P,Q,Z,D,L);
}
```

문법이 맞으면 의미 동작이 실행되고 Select 구조가 만들어집니다.

가독성을 위해 원본 규칙의 줄바꿈만 조정했습니다.

매개변수 표현식과 번호

parse.y:1204-1208의 VARIABLE 규칙입니다.

```
expr(A) ::= VARIABLE(X). {  
    if( !(X.z[0]=='#' && sqlite3Isdigit(X.z[1])) ){  
        u32 n = X.n;  
        A = tokenExpr(pParse, TK_VARIABLE, X);  
        sqlite3ExprAssignVarNumber(pParse, A, n);  
    }
```

표현식을 만들고, 바인딩에 사용할 매개변수 번호를 정합니다.

규칙 앞부분 발췌입니다. 뒤의 내부 #N 처리 분기와 닫는 부분은 생략했습니다.

번호는 어떻게 정해질까

SQL 안의 표현	번호 규칙	expr.c 위치
?	이전 최대 번호의 다음 번호	1328-1331
?1, ?2	명시한 정수를 번호로 사용	1334-1361
:name 반복	같은 이름이면 기존 번호 재사용	1363-1375

바인딩되지 않은 매개변수의 값은 NULL입니다.

bind가 값을 저장하는 곳

vdbeapi.c:1791-1798의 정수 바인딩 핵심입니다.

```
Vdbe *p = (Vdbe *)pStmt;
rc = vdbeUnbind(p, (u32)(i-1));
if( rc==SQLITE_OK ){
    /* 범위 검증 assert 생략 */
    sqlite3VdbeMemSetInt64(&p->aVar[i-1], iValue);
}
```

1부터 시작하는 API 인덱스 *i*를 배열 인덱스 *i-1*로 바꿉니다.

파서가 정한 번호와 aVar의 값 저장이 여기서 연결됩니다.

실제 Parser 추적 실행

고정된 SELECT를 준비하고, step은 호출하지 않습니다.

```
./build/student_lab data/week3.db parser ok  
./build/student_lab data/week3.db parser syntax  
./build/student_lab data/week3.db parser name
```

ok는 준비 성공, syntax와 name은 의도한 실패 사례입니다.

추적 조건: SQLITE_DEBUG 빌드와 PRAGMA parser_trace=ON

Shift와 Reduce 읽기

parser ok의 실제 출력 일부입니다.

```
parser: Input 'SELECT' in state 0
parser: Shift 'SELECT', go to state 329
parser: Input 'ID' in state 329
parser: Reduce 100 [distinct ::=].
...
PREPARE rc=0 parameters=1
```

Shift는 입력 토큰을 받아들이고, Reduce는 문법 규칙을 적용합니다.

상태 번호와 규칙 번호를 외우기보다 토큰과 규칙 이름을 연결합니다.

두 가지 준비 실패

비교	syntax 모드	name 모드
SQL	SELECT FROM student;	SELECT missing_column FROM student;
문제	SELECT 목록이 없음	열 이름을 찾을 수 없음
관찰	Syntax Error!	no such column
단계	문법 처리 실패	이름 해석 실패

둘 다 prepare 오류이지만, 이름 해석 오류를 문법 오류로 부르지는 않습니다.

소스 분석 확인 과제

과제 1

?1이 TK_VARIABLE이 되는 파일과 줄을 찾습니다.

과제 2

parse.y에서 매개변수 번호 결정 함수를 찾아 호출을 따라갑니다.

과제 3

토큰 ?1과 바인딩 값 2001이 다른 시점에 처리됨을 설명합니다.

추가 관찰: ./build/token_probe "SELECT '한글';"에서 문자열의 바이트 길이는?

별도 분석 과제 제출

토큰과 문법

?1을 인식하는 위치와 VARIABLE 규칙을 설명합니다.

번호와 값

매개변수 번호 결정과 실제 값 저장 시점을 비교합니다.

오류와 호출 경로

문법·이름 해석 오류와 prepare 호출 경로를 기록합니다.

제출 파일: 학번_parser.md · 3-tier 실습과 별도 제출